

Deep Reinforcement Learning for Traffic Light Control

Ben Bekir Ertugrul
School of Engineering
Jönköping University
Jönköping, Sweden

Abstract—This document is a model and instructions for L^AT_EX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.

Index Terms—component, formatting, style, styling, insert.

I. INTRODUCTION

- What are SUMO limitations? - Do we optimize traffic lights only for cars or also bikes, pedestrians, etc. - Do we focus on multiple intersections or just one? - Are we using acyclical or cyclical light switches? May one of them be more realistic than the other? - Do we train on multiple scenarios, like “rush hours” and low traffic conditions, or do we only have normal traffic? - Do we use DQN or are there also other choices like PPO (is PPO able to combine with neural nets?)?

a good starting tutorial on how to generate traffic scenarios can be found here [1].

- at the end, compare normal (static) traffic light with our solution. if possible, the static traffic light control should be decently good. maybe its possible to use a real example?

II. NETWORKS

A. Jönköping

map of sweden downloaded here:
<https://download.geofabrik.de/europe/sweden.html> geojson
polygon object created here: <https://geojson.io/> chosen
geojson found here¹.

then, we used osmium and osmfilter to apply the polygon shape to the map of sweden. afterwards, we filter only for highways, except for those of the following types:

- footway
- path
- pedestrian
- steps
- cycleway
- bridleway

finally, we use the SUMO *netconvert* tool to convert the map into a SUMO network.

as for limitations, we extract lanes, intersections, and traffic lights, but do not have information about the speed limits on the roads. therefore, our simulation will treat every road the same.

¹<https://github.com/benbekir/traffic-rl/blob/main/simulations/networks/jkpg/artifacts/jkpg.geojson>



Fig. 1. The filter (in grey) that defines the boundaries of the road network in Jönköping.



Fig. 2. The network as shown in the SUMO nededit UI, highlighting the traffic light placements.

1) *Demand Modeling*: since there is no publicly available data regarding traffic assignment zones and demand in jönköping, we use the SUMO tool randomTrips² with the random seed set to 12 to randomly create trips. we create 3600 individual trips in total where a new trip is spawned into the simulation on every timestep.

2) *Rush Hours and Noise*: im not sure about this yet

²<https://sumo.dlr.de/docs/Tools/Trip.html#randomtripspy>

3) *Detectors*: To retrieve information that our agents can use, there are a couple of detector options that we could add to our environment (see https://sumo.dlr.de/docs/Simulation/Output/#simulated_detectors).

- Inductive loop detectors
- Instant induction loops
- Lane area detectors
- Multi-Entry-Exit detectors
- Route detectors

we decide to use lane area detectors since they simulate vehicle tracking cameras, which gain popularity in the real world (i think in china especially) and provide rich information. these detectors are placed on every lane at every intersection that has a traffic light. the maximum length of the recorded lane segment is 25 meter.

REFERENCES

- [1] D. A. Guastella, E. Montero-Porras, A. Morales-Hernández, and G. Bontempi, "Traffic modeling with sumo: a tutorial," 2025. [Online]. Available: <https://arxiv.org/abs/2304.05982>